B14 Experiment tracking with MLflow

Tracking, registry and the model lifecycle.

B14-L1 Track every run with MLflow

One run per fit, inside a with block, with parameters, commit, data hash, metrics and the model.

```python
import mlflow

mlflow.set_experiment("churn")
with mlflow.start_run(run_name="lr-c1"):
    mlflow.log_params({"C": 1.0, "seed": 7, "features": "v3"})
    mlflow.set_tags({"git_commit": commit, "input_sha256": data_hash})
    model = train(X_train, y_train, C=1.0, seed=7)
    mlflow.log_metric("cv_auc", cv_auc)
    mlflow.sklearn.log_model(model, name="model", input_example=X_train[:5])
```

What to log per run	Params (incl. seed), tags (git_commit, input_sha256), metrics, the model.
MLflow parameters	Stored as strings and immutable within a run.
Why with mlflow.start_run()	The run ends with FAILED status if training raises, instead of staying open.
log_model in MLflow 3	mlflow.sklearn.log_model(model, name="model"), with pinned requirements saved alongside.

B14-L2 Registry versions and aliases, not stages

Versions are fixed. Aliases move. Promote by comparison on the same data and metric, and keep the previous champion addressable.

```python
from mlflow import MlflowClient

client = MlflowClient()
client.set_registered_model_alias("churn_model", "champion", "3")
model = mlflow.pyfunc.load_model("models:/churn_model@champion")
```

Version vs alias	A version is fixed. An alias is a movable label on a version.
Load the current production model	mlflow.pyfunc.load_model("models:/name@champion").
Model stages	Deprecated since MLflow 2.9, not supported in Unity Catalog. Use aliases.
Safe promotion	Compare on the same data and metric, require a margin, keep previous_champion for rollback.

B14-L3 Reproduce a run from its record

Code from the commit, environment from the lock, data checked by hash, settings converted back to types, metric compared within a tolerance.

```python
run = mlflow.get_run(run_id)
run.data.params            # {'C': '1.0', 'seed': '7', 'features': 'v3'}  strings
run.data.tags["git_commit"], run.data.tags["input_sha256"]
run.data.metrics["cv_auc"]
reqs = mlflow.pyfunc.get_model_dependencies(f"runs:/{run_id}/model")   # path to
  requirements.txt
```

Reproduce a run	Commit, environment from lock, data verified by hash, typed params, rerun, compare within tolerance.
Parameters from mlflow.get_run	Strings. Convert with the expected types before use.
Model dependencies	mlflow.pyfunc.get_model_dependencies(model_uri) returns the path to requirements.txt.
Rerun differs beyond tolerance	Something that changes results was not logged. Find it and log it.